

Chap.10 트랜스포머 (Transformer)

방 수 식 교수
(bang@tukorea.ac.kr)

한국공학대학교 전자공학부

2024년도 2학기
고급머신러닝실습 & 인공지능설계실습2

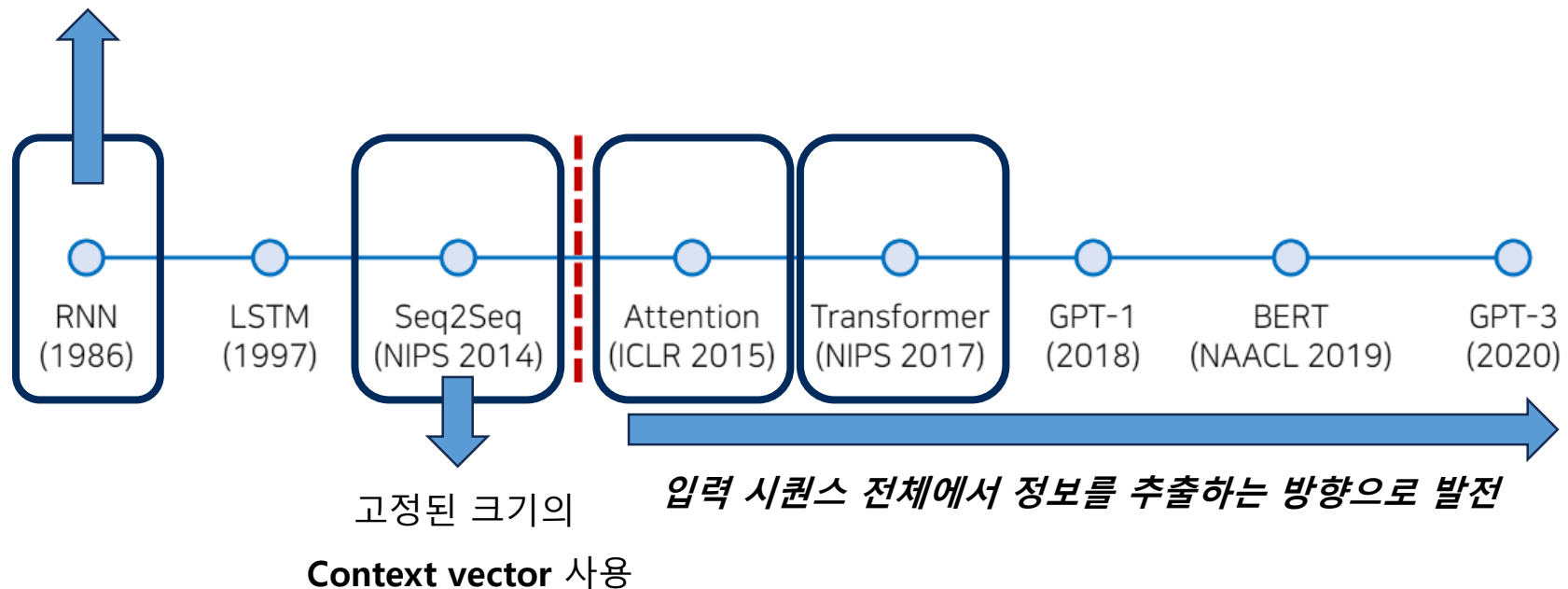
트랜스포머의 개요



■ 트랜스포머(Transformer) 모델 등장 배경

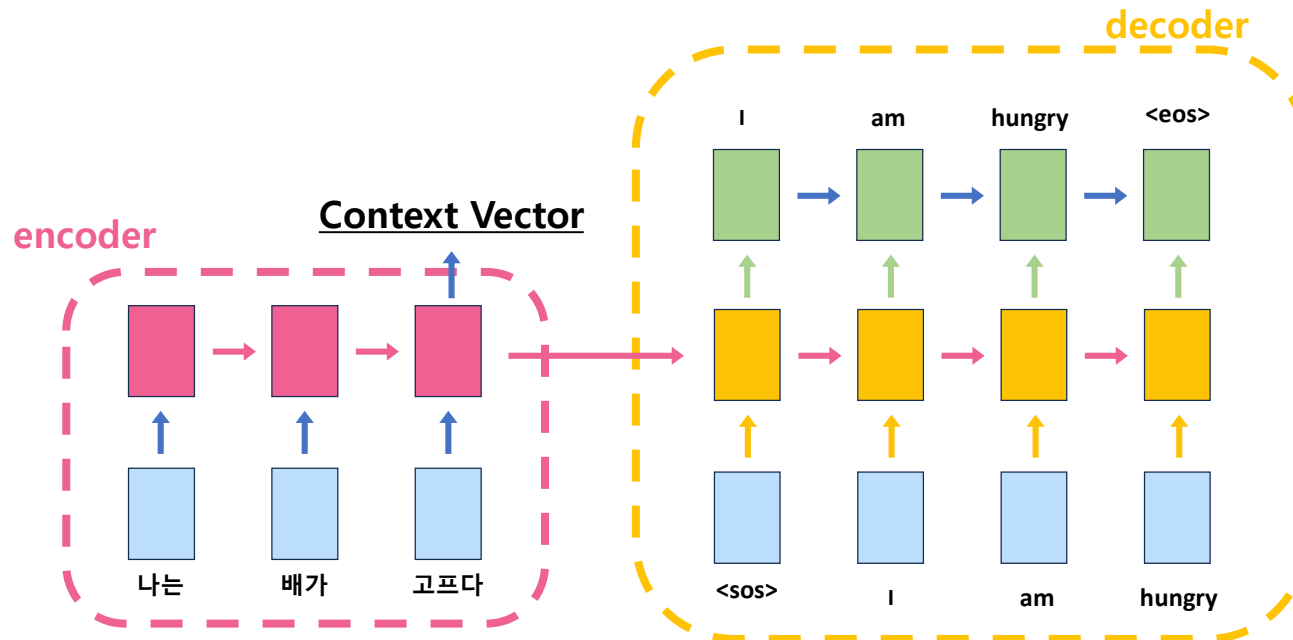
RNN의 한계점 "장기 의존성" 문제

데이터의 길이가 길어질수록 앞의 내용이 뒤로 전달되지 않는 문제 발생



▪ Seq2Seq란?

- 입력 시퀀스를 받아 다른 형태의 출력 시퀀스로 변환 구조
- 인코더로 입력 정보를 압축한 **Context Vector**로 Decoder가 다음 토큰 예측
- 서로 다른 두 개의 RNN 사용(서로 다른 Weight, Loss는 공유)



Context Vector에 마지막 단어의 정보가 가장 뚜렷하게 담김

→ 문맥적으로 중요한 부분을 강조할 수 있는가?

Attention Mechanism

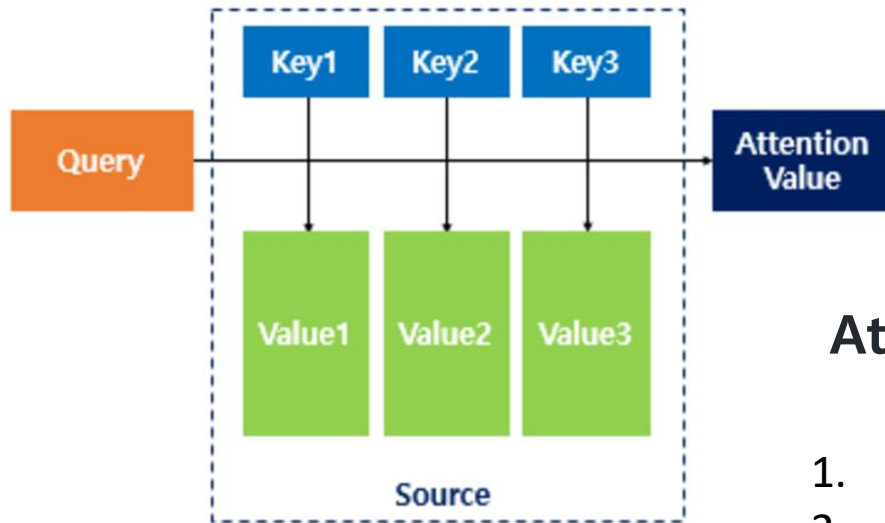


■ Attention Mechanism

- 디코더에서 출력 단어를 예측하는 매 시점(time step)마다, 인코더에서의 전체 입력 문장을 다시 한 번 참고

```
dict = {"2019": "Transformer", "2020": "BERT"}  
print(dict["2019"]) => ?
```

Key Value
Query



$\text{Attention}(Q, K, V) = \text{Attention Value}$

1. Q와 K의 유사도 => Score
2. Score를 Value에 맵핑
3. 맵핑된 값을 모두 더함 => Attention Value

Attention Mechanism

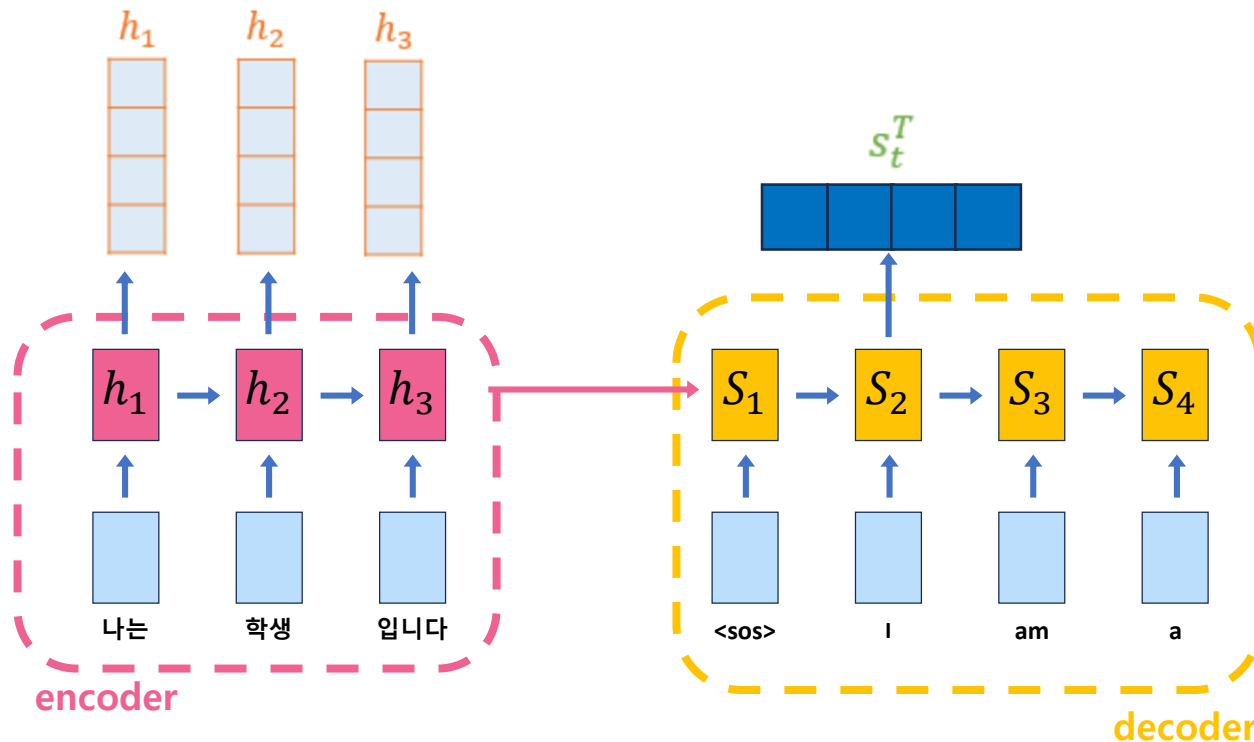


▪ Dot-Product Attention *인코더의 “모든 시점” 정보를 참조 => 디코더 출력 단어를 생성*

- Attention Mechanism 중 가장 이해가 쉬움. (타 Attention 과 거의 유사)

<Step 1>

- **Query** : s_t (t 시점의 디코더 셀에서의 은닉 상태)
- **Key** : $[h_1 \ h_2 \ \cdots \ h_T]$ (모든 시점의 인코더 셀의 은닉 상태들)
- **Value** : $[h_1 \ h_2 \ \cdots \ h_T]$ (모든 시점의 인코더 셀의 은닉 상태들)



Attention Mechanism

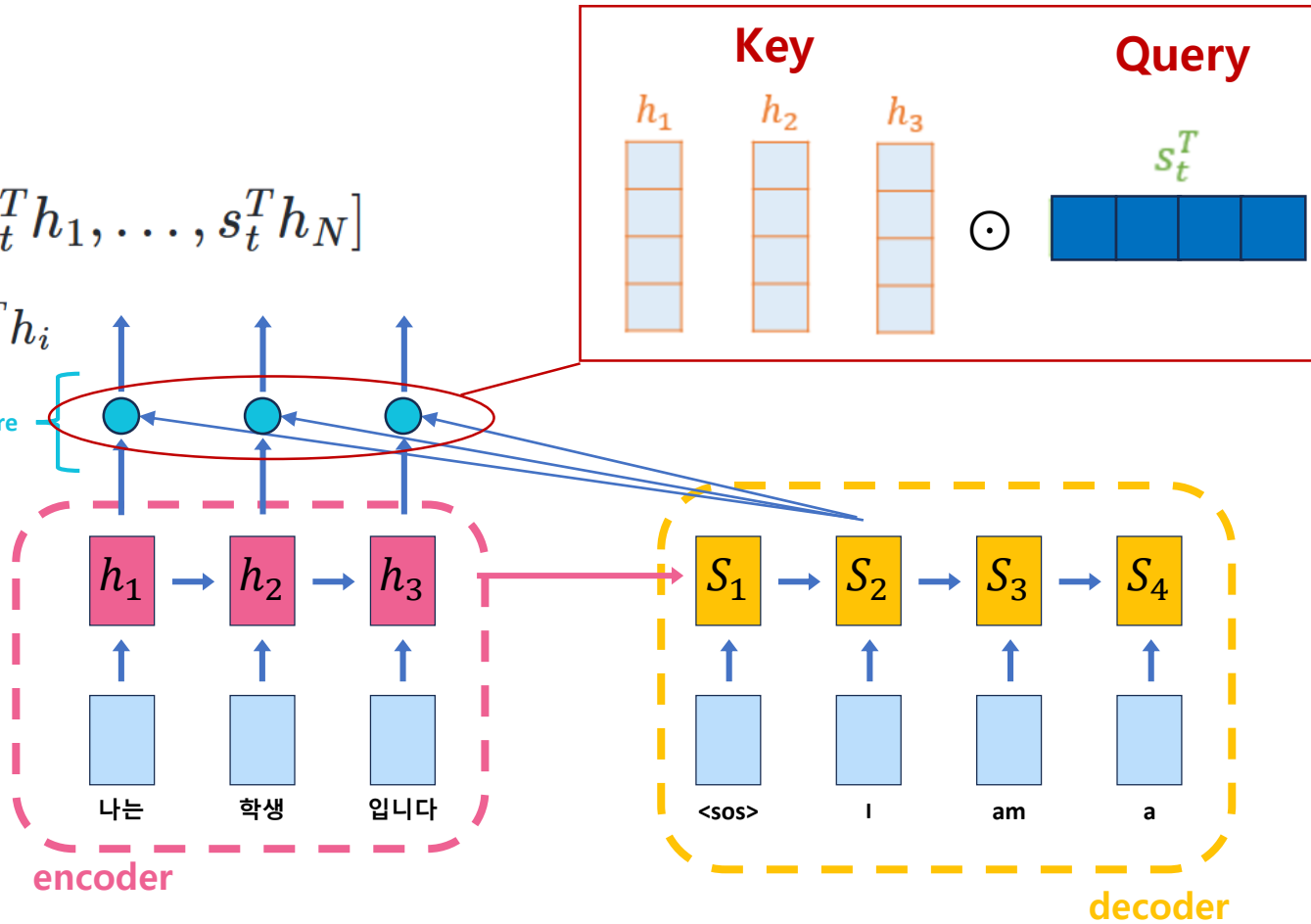


<Step 2>

$$e^t = [s_t^T h_1, \dots, s_t^T h_N]$$

$$\text{score}(s_t, h_i) = s_t^T h_i$$

Attention Score

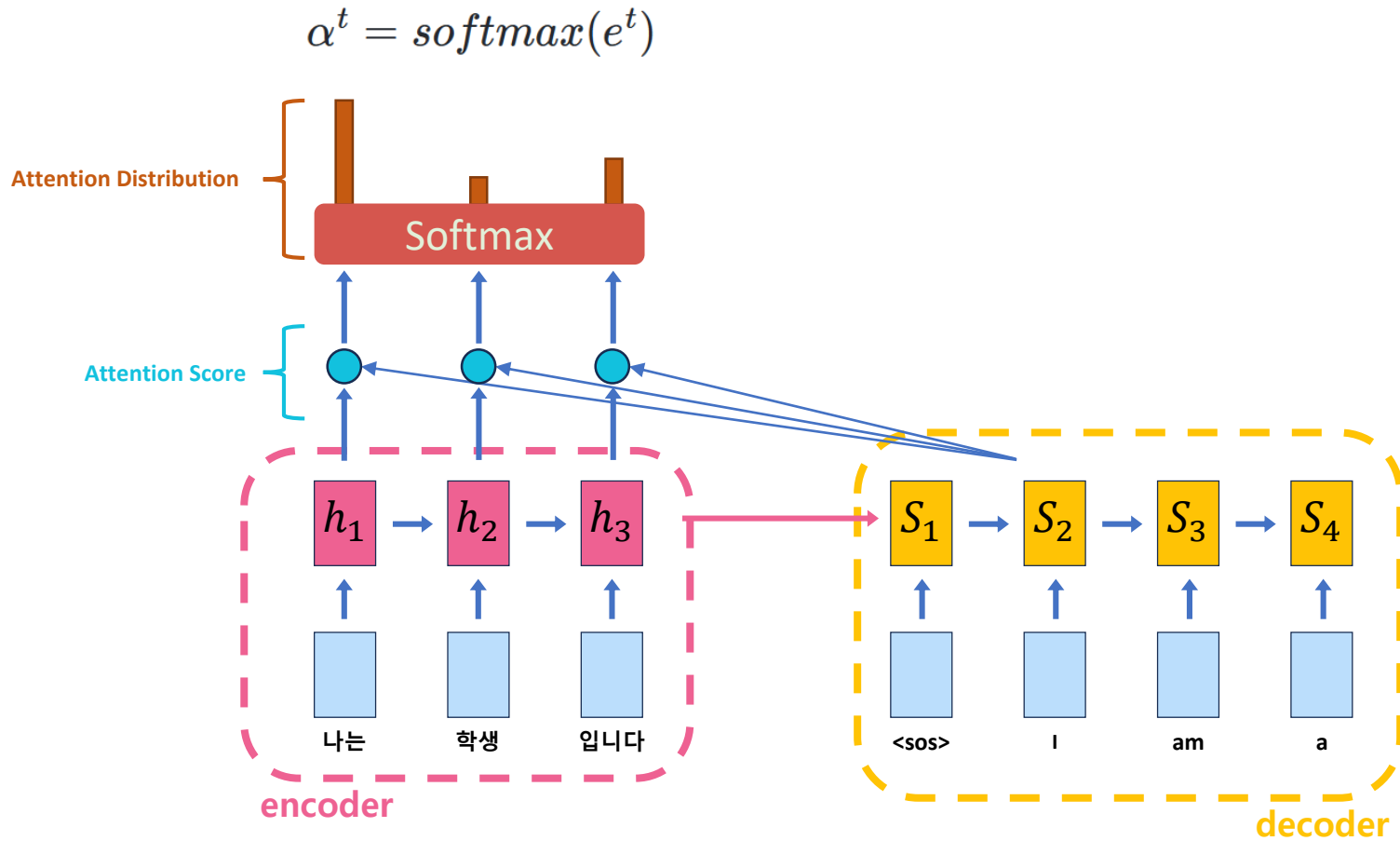


<Dot-Product Attention>

Attention Mechanism



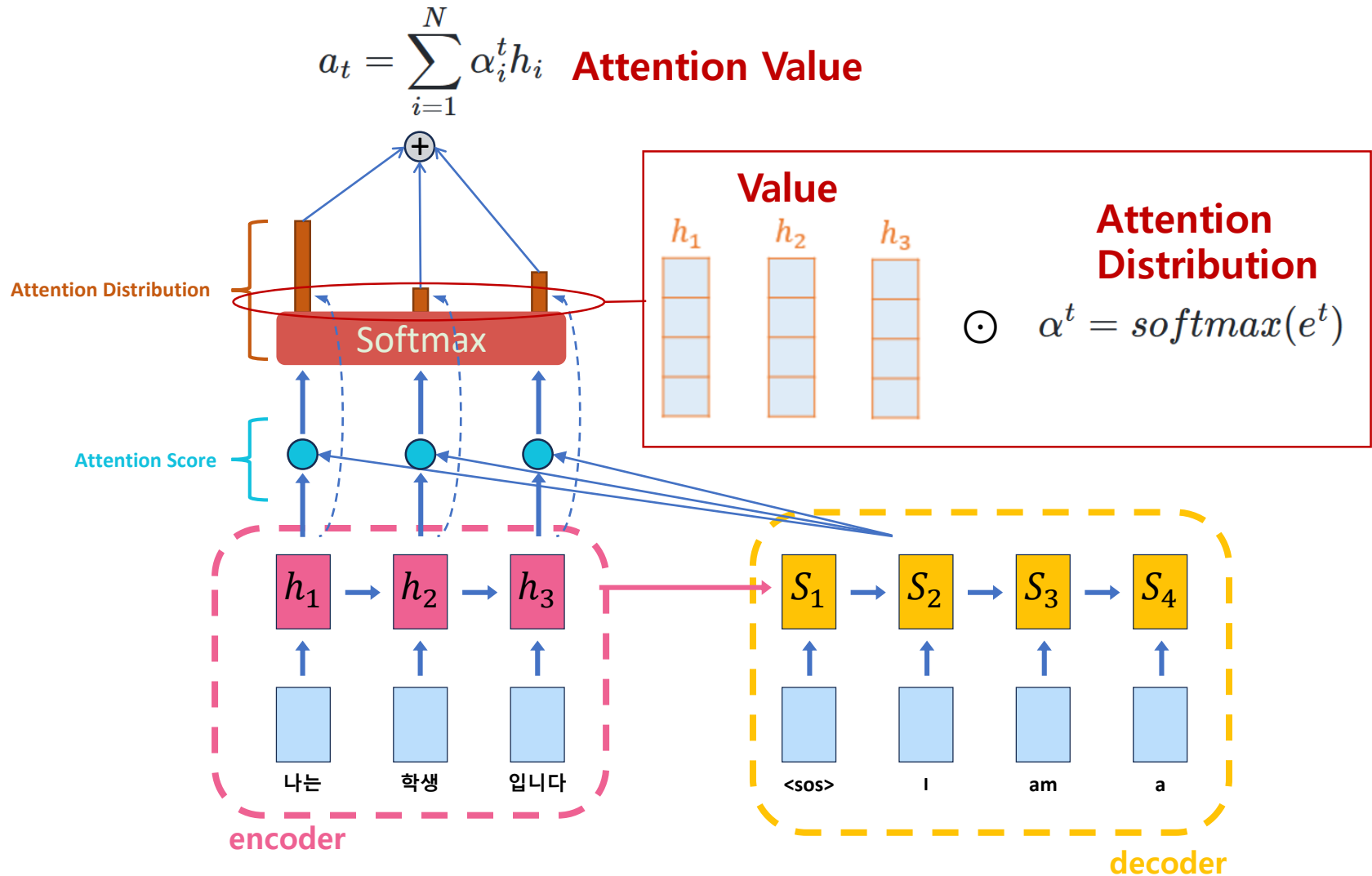
<Step 3>



Attention Mechanism



<Step 4>

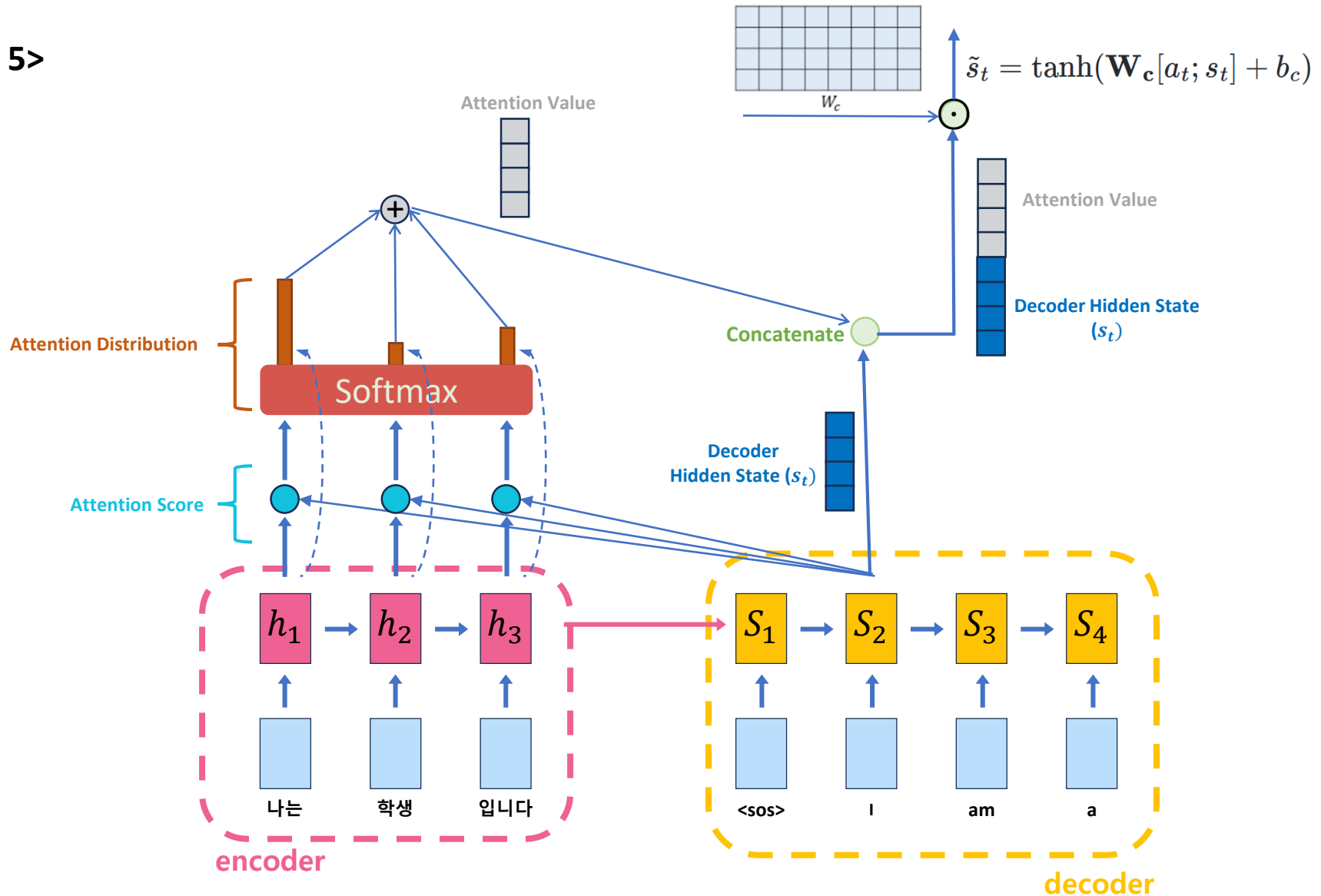


<Dot-Product Attention>

Attention Mechanism



<Step 5>

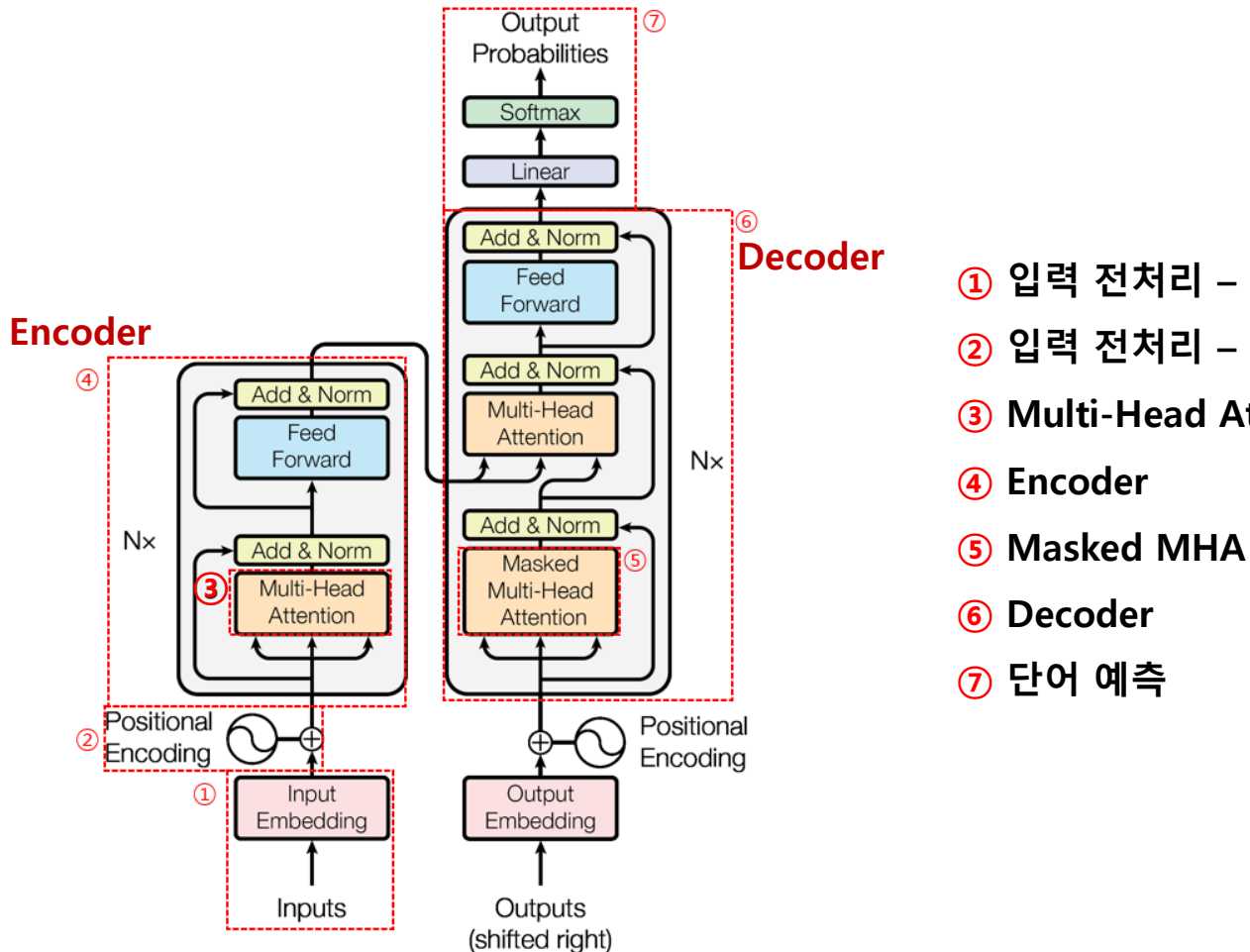


<Dot-Product Attention>

Transformer Model Architecture



Overview



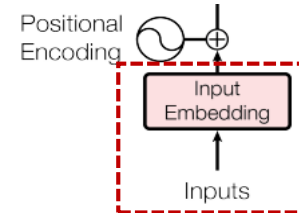
- ① 입력 전처리 – 1 (Tokenization & Embedding)
- ② 입력 전처리 – 2 (Positional Encoding)
- ③ Multi-Head Attention (MHA)
- ④ Encoder
- ⑤ Masked MHA
- ⑥ Decoder
- ⑦ 단어 예측

① 입력 전처리- Tokenization



▪ Token

- 자연어 처리(NLP)와 기계 학습에서
텍스트를 작은 단위로 나누는 과정에서 생성되는 개별적인 요소를 의미



▪ Tokenization

- 텍스트에서 토큰(token)이라 불리는 단위로 나누는 작업

"Tomatoes are one of the most popular plants for vegetable gardens."

Text

↓ Tokenization

["Tomatoes", "are", "one", "of", "the", ..., "for", "vegetable", "gardens", "."]

단어 단위

미등록 토큰 문제

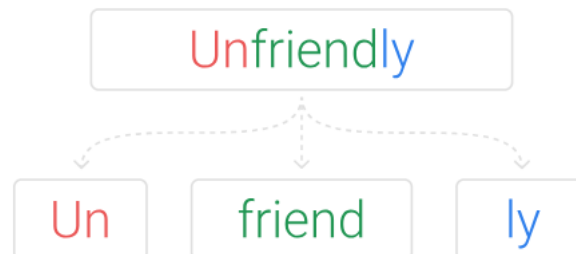
["T", "o", "m", "a", "t", "o", "e", "s", ..., "g", "a", "r", "d", "e", "n", "s", "."]

문자 단위

시퀀스 길이 비효율성

▪ Sub-word Tokenizer:

- 하나의 단어를 여러 서브 워드로 분리

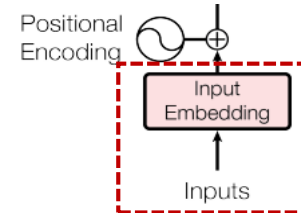


① 입력 전처리- Embedding



■ Embedding

- 사람이 쓰는 자연어를 기계가 이해할 수 있는 숫자의 나열인 **벡터**로 바꾼 결과



2차원 텐서 (토큰 길이 X 문장 개수)

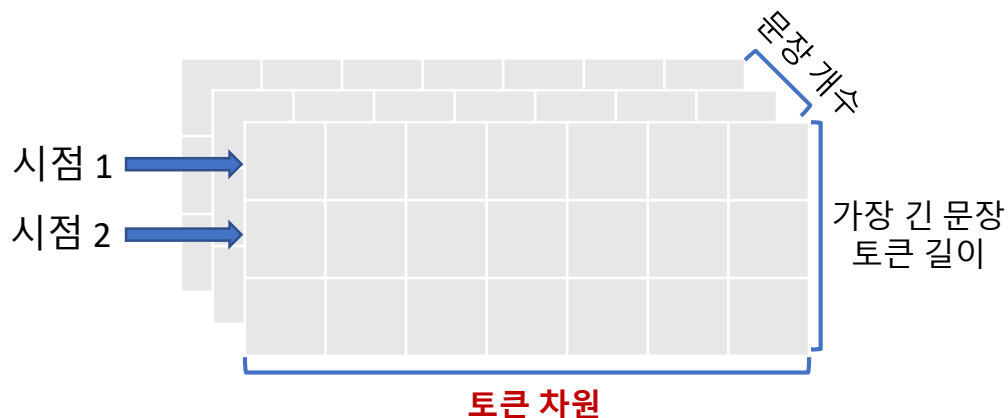
1.	T	omato	e	s	are	one	...	for	vegetable	garden	.
2.	T	omato	plants	come	...	of	sizes	.	<padd>	<padd>	<padd>

Embedding

3.487
7.291
0.562
...
4.776

가장 긴 문장 기준 padding 처리

3차원 텐서 생성

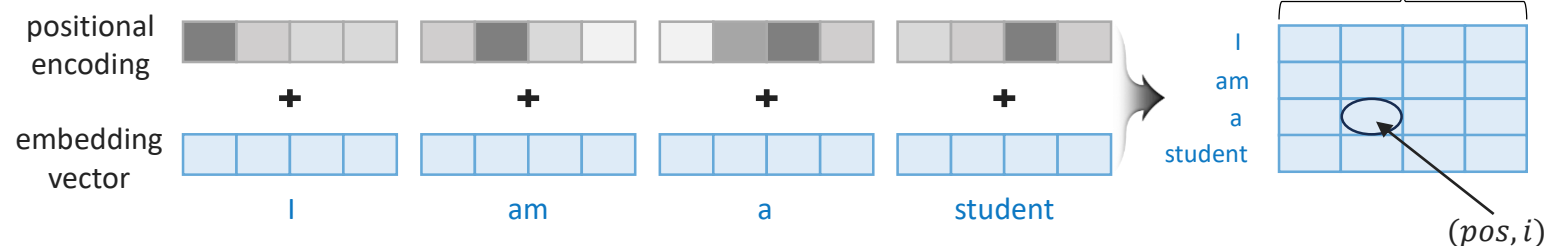
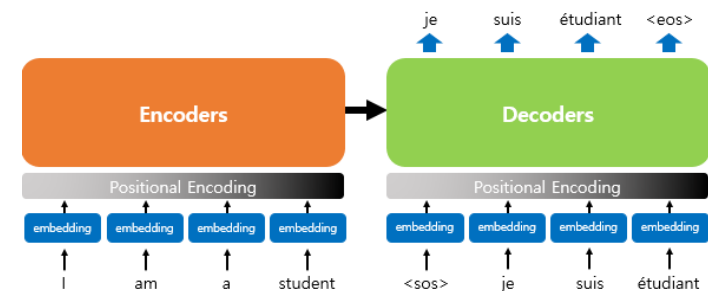


② 입력 전처리- Positional Encoding



Positional Encoding이란?

- 위치의 **Representation**을 벡터로 표현한 것
- Embedding Vector에 **위치 정보를 더해 모델의 입력**으로 사용

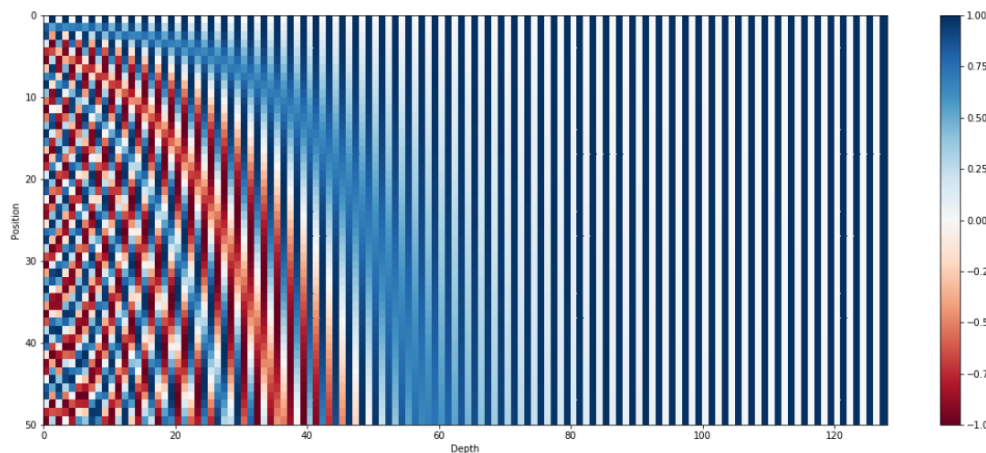


$$PE_{(pos, 2i)} = \sin(pos/10000^{2i/d_{model}})$$

$$PE_{(pos, 2i+1)} = \cos(pos/10000^{2i/d_{model}})$$



주기성을 이용하여
위치 정보를 결정



② 입력 전처리- Positional Encoding



Positional Encoding의 필요성

“Tomatoes are one of the most popular plants for vegetable gardens.”



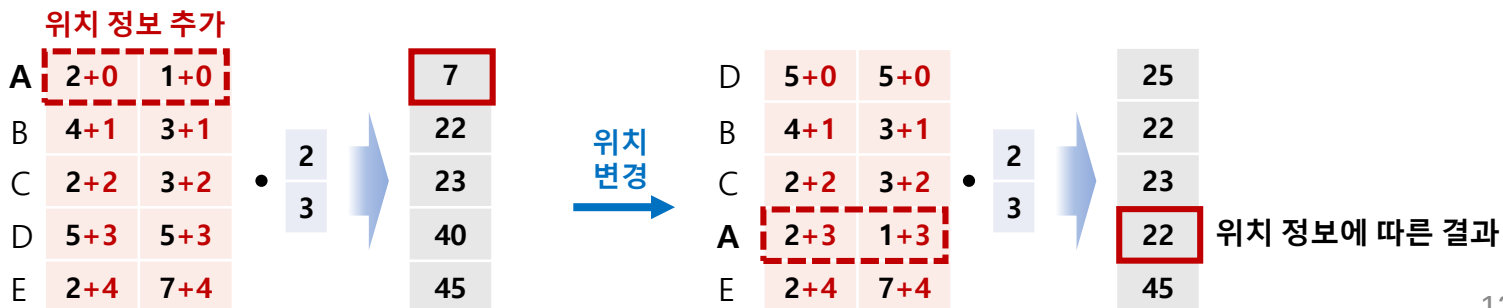
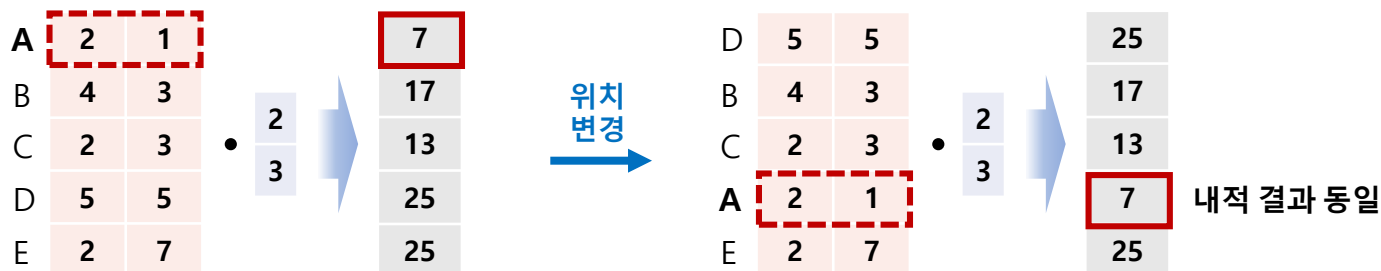
그대로 입력 시

“Tomatoes are one of the most popular **vegetable** for **plants** gardens.”

“**vegetable** are one of the most popular **Tomatoes** for **plants** gardens.”

위치에 대한 정보가 없다면 해석 오류 발생

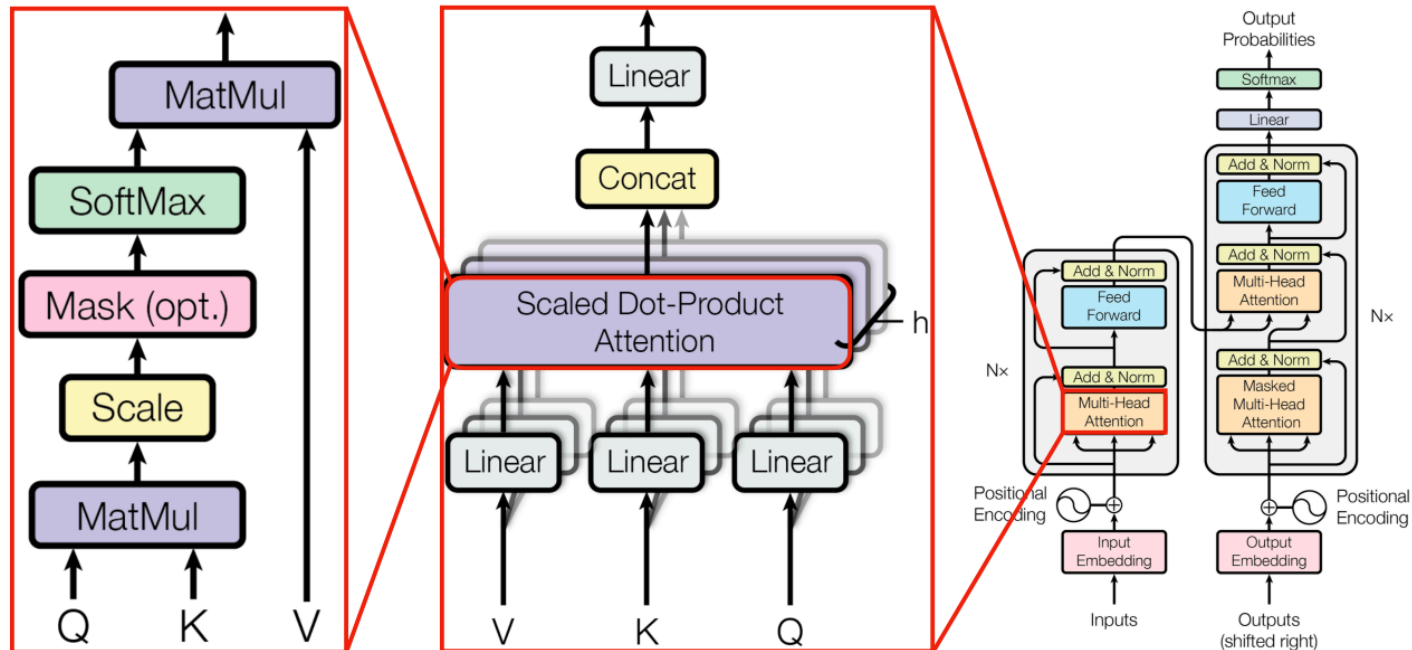
개념적 이해



③ Multi-Head Attention: Self-Attention



▪ Self-Attention

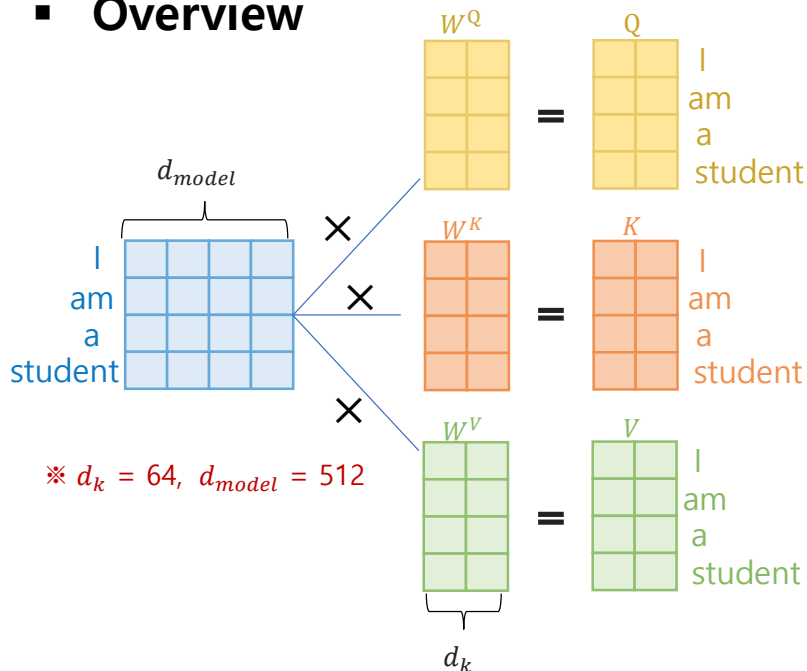


Self-Attention

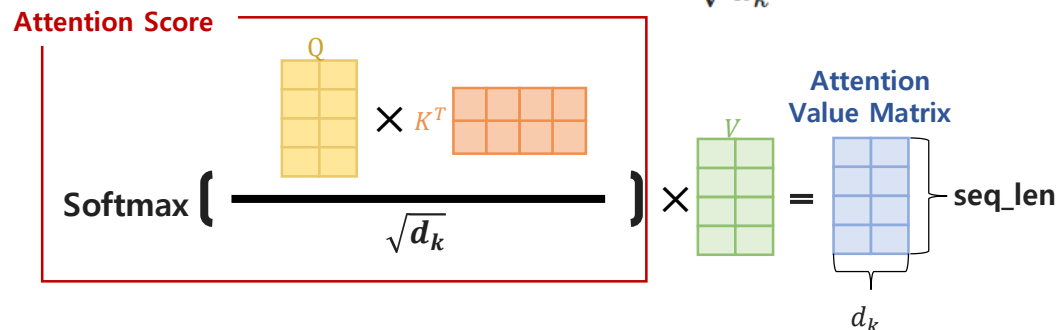
Self-attention



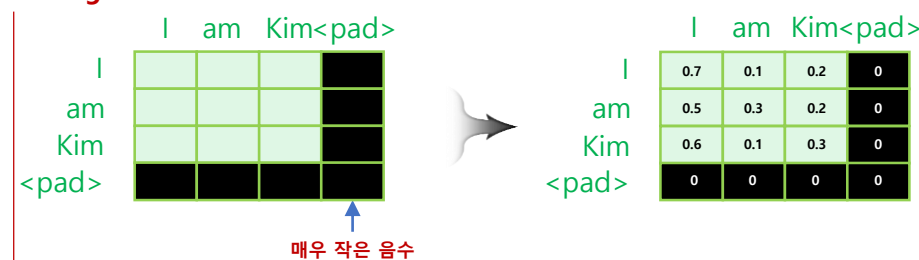
Overview



$$Attention(Q, K, V) = softmax\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$



Padding Mask



1. Linear Transformation

Input Matrix에 가중치 Matrix 내적 → Query, Key, Value 벡터로 변환

2. Scaled Dot-Product Attention

Scaled Dot-Product Attention 적용 후 Value와 내적 → Attention Value Matrix 계산

3. Padding Mask

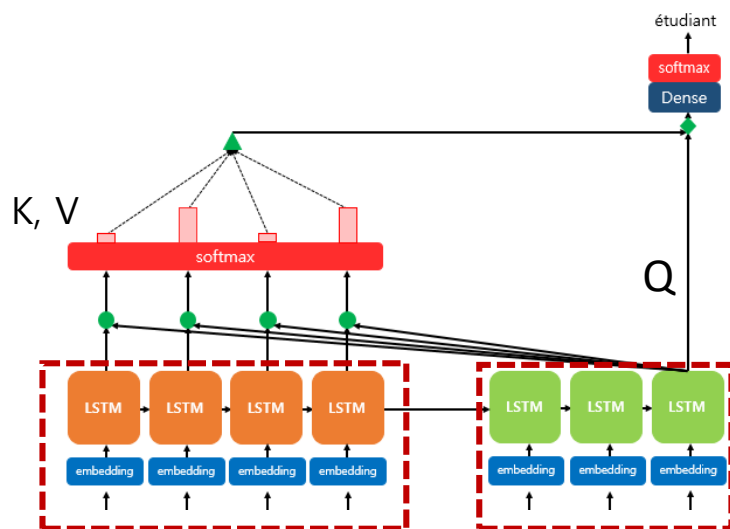
입력 시퀀스의 길이가 짧을 경우 Pad 토큰을 추가해 시퀀스 길이 맞춤

Attention Mechanism 과의 비교



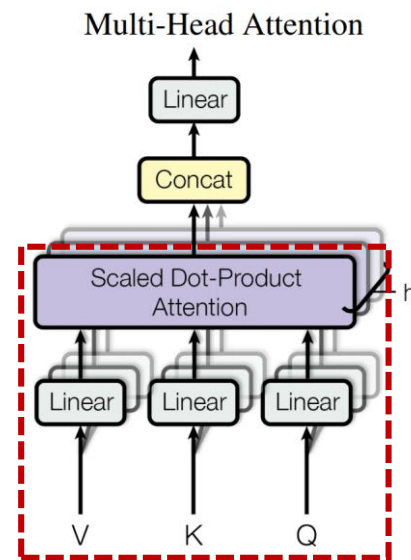
지능형 예측·진단 연구실
Intelligent Prognostics & Diagnostics Lab.

• 기존 Attention



Q, K & V의 입력(디코더 / 인코더)이 다름
Q 시점에서 K, V는 고정됨

• Self Attention

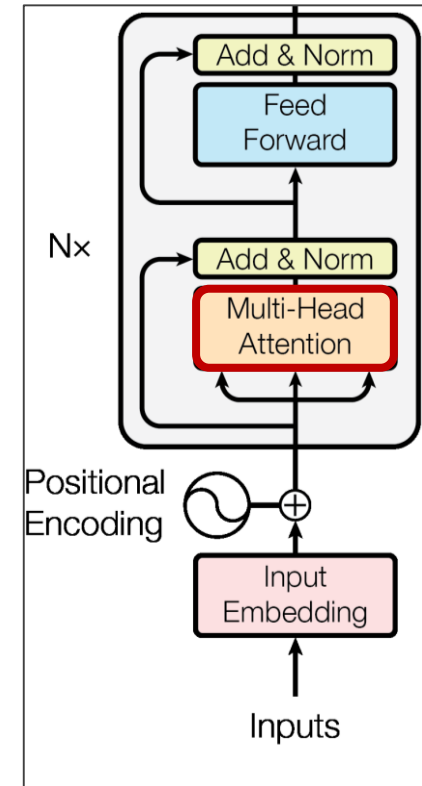


Q, K, V가 동일 입력 및 동일 시점 업데이트

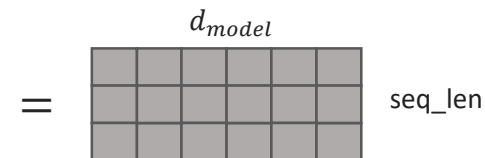
+) 두 매커니즘은 적용 범위에 차이가 있다는 점에 유의

지능형 예측·진단 연구실
Intelligent Prognostics & Diagnostics Lab.

The diagram illustrates the Multi-head Attention mechanism. At the top, the input sequence is represented as a matrix of size $seq_len \times d_{model}$. This input is projected by h parallel linear heads (Linear head 1, ..., Linear head $h-1$, Linear head h). Each head i outputs a matrix of size $seq_len \times d_k$, where d_k is the dimension of each head. The first head's outputs are labeled Q (Query), K (Key), and V (Value). The Q and K matrices are used to calculate the attention weights via the formula: $Attention(Q, K, V) = softmax(\frac{QK^T}{\sqrt{d_k}})V$. The result of this calculation is the Attention Value Matrix, also of size $seq_len \times d_k$. The outputs from all h heads are then concatenated to form the final output matrix of size $seq_len \times (h \times d_k)$. A red text annotation "Head의 개수 h 는?" (Number of heads h is?) is present near the heads. Vertical text boxes labeled "동일과정" (Same process) indicate that the same mechanism is repeated for each head.

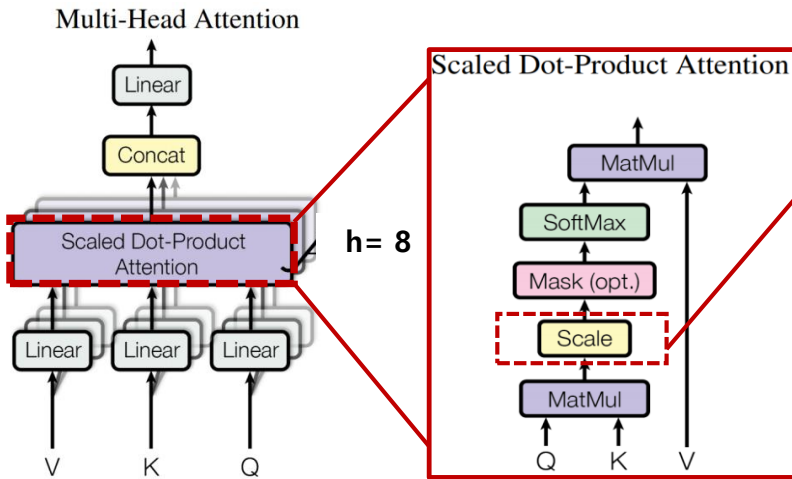


Parallel Attention Matrix



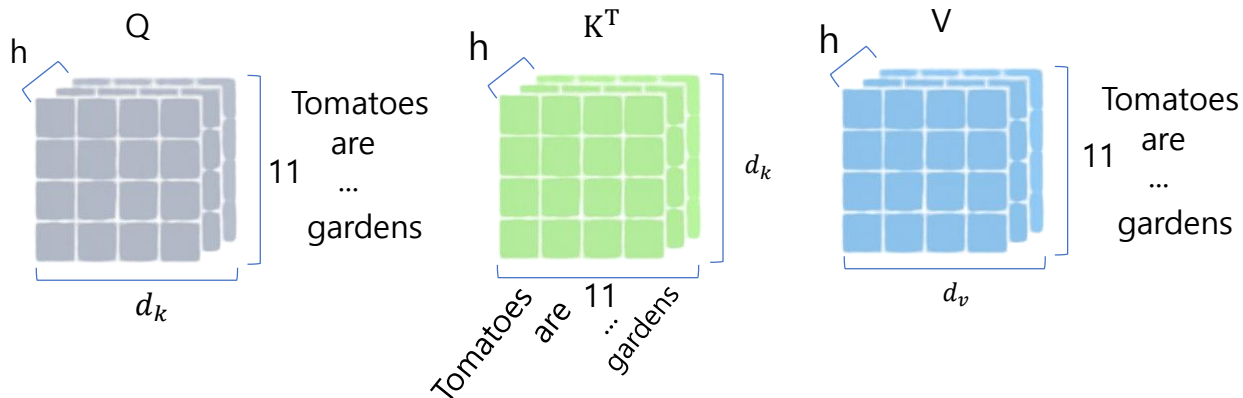
Multi-Head Attention

▪ Scaled Dot-Product Attention 구조



내적 값의 편차로 인한 학습 영향을 줄이기 위해 softmax 전 scaling을 적용 (뒤에서 자세히 설명)

1. Q, K^T 내적
2. Scaling 이후 softmax (열 기준)
3. Value 내적한 최종 값

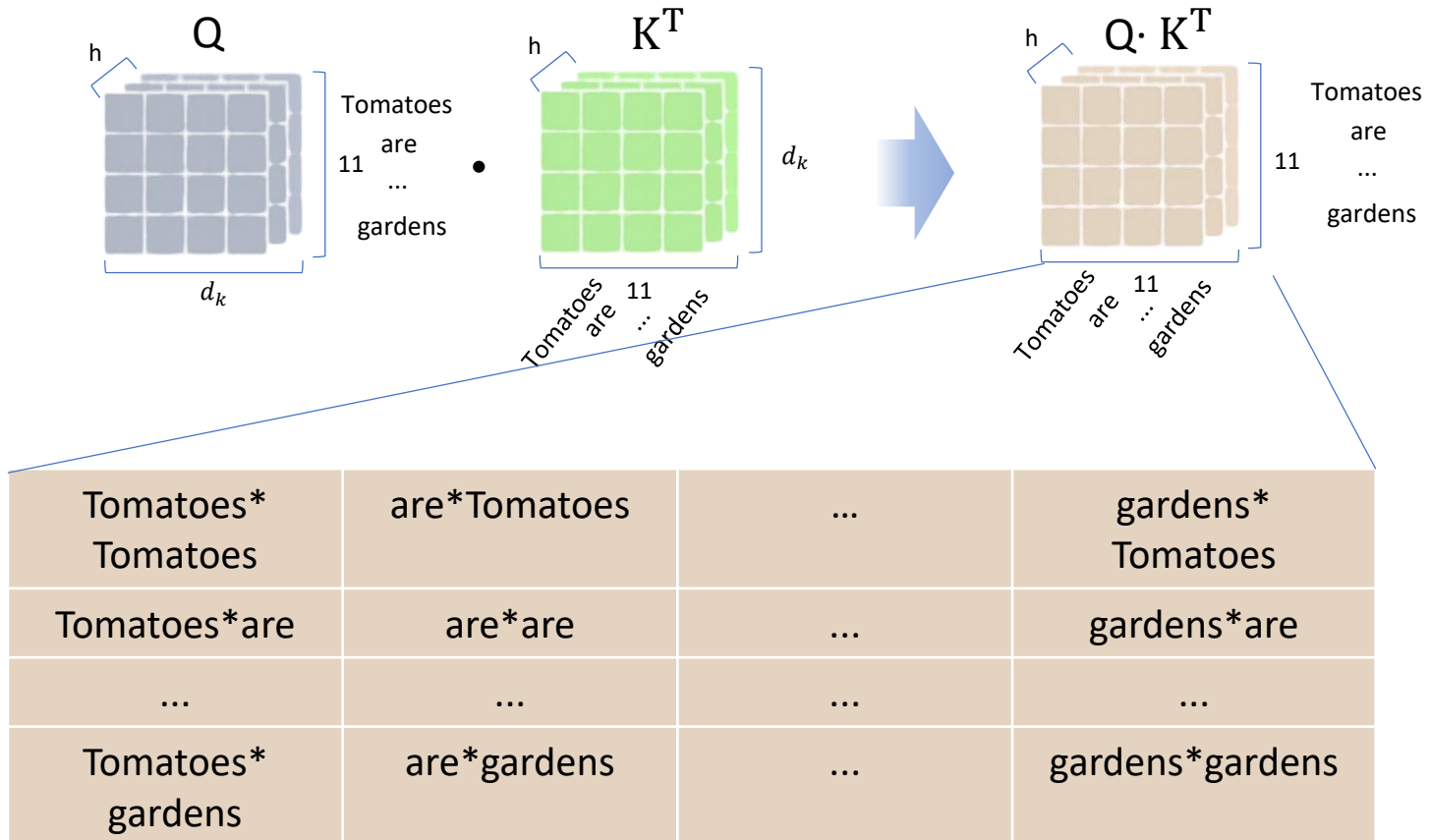


Multi-Head Attention



1. Q, K^T 내적

$$Attention(Q, K, V) = softmax(\frac{QK^T}{\sqrt{d_k}})V$$



두 내적의 결과는 문장의 각 단어에 대한 관련성 정보 내포

Multi-Head Attention



2. Scaling 이후 softmax (열 기준)

- Scaling의 이유?

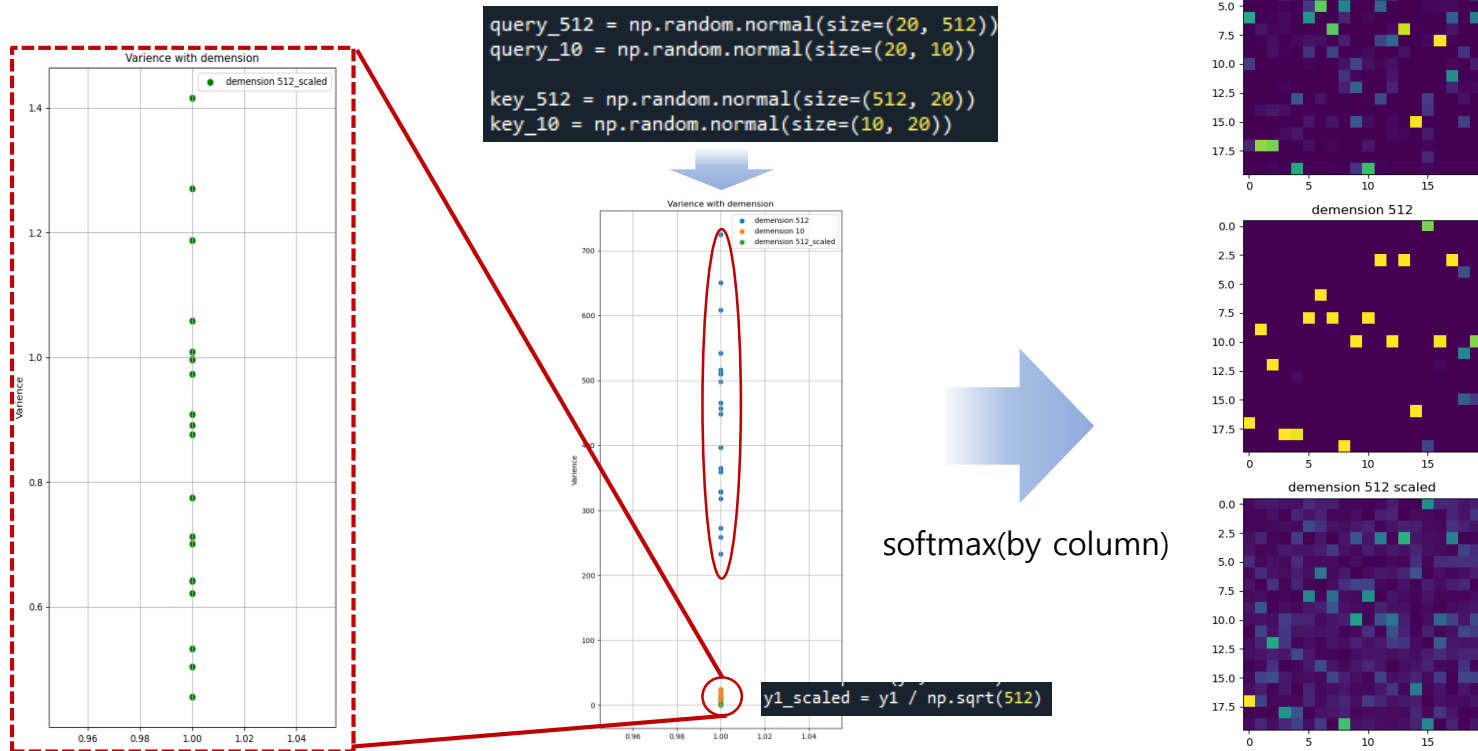
차원이 커짐에 따라 내적 결과값 간의 편차(분산)가 높아짐

이로 인해 softmax 결과가 가장 큰 값을 제외 시 나머지는 거의 0에 수렴

⇒ 효율적인 학습 불가

⇒ Scaling을 통해 다양한 변수를 고려한 학습 보장

$$Attention(Q, K, V) = softmax(\frac{QK^T}{\sqrt{d_k}})V$$

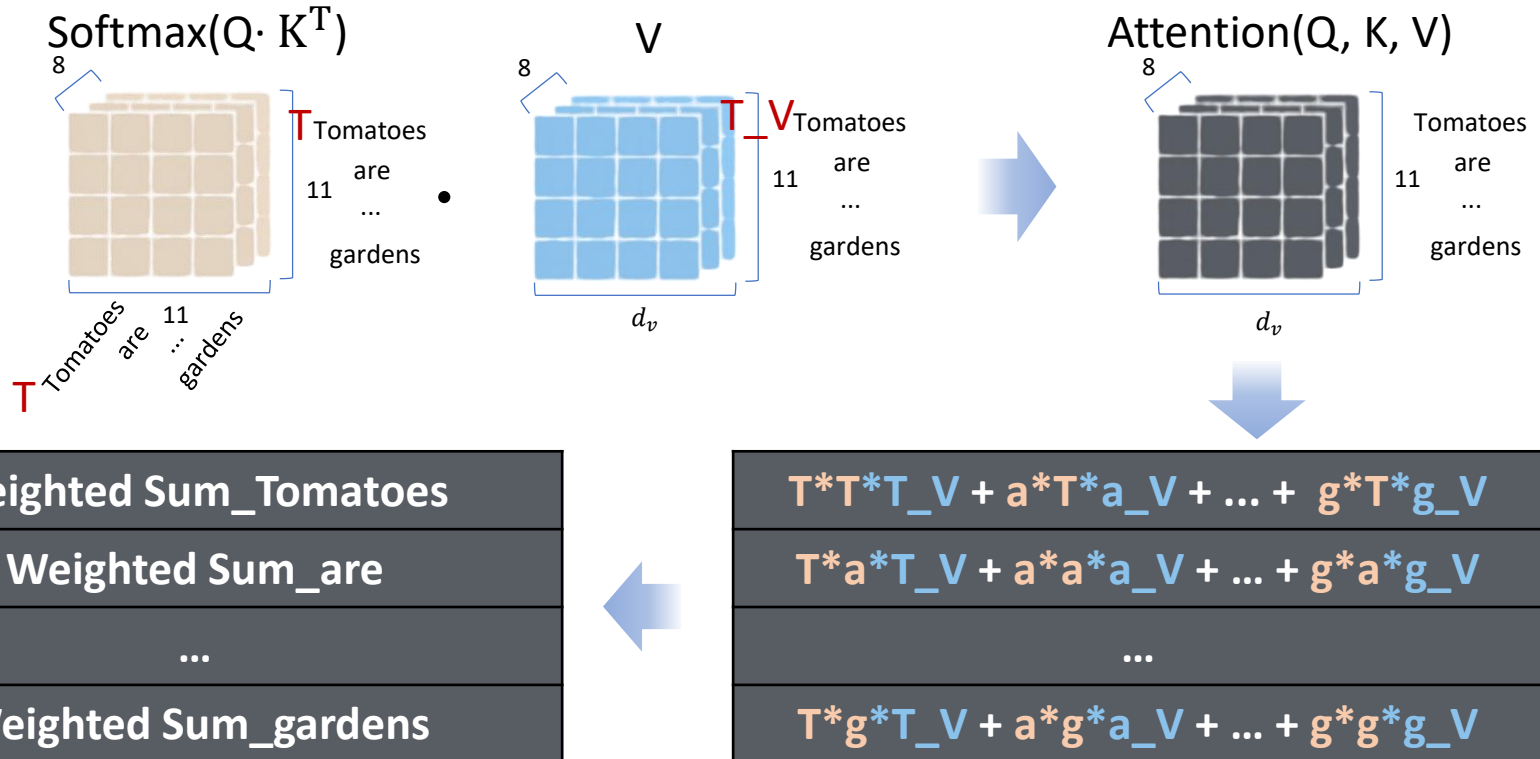


Multi-Head Attention



3. V와의 Dot-product

$$Attention(Q, K, V) = softmax(\frac{QK^T}{\sqrt{d_k}})V$$

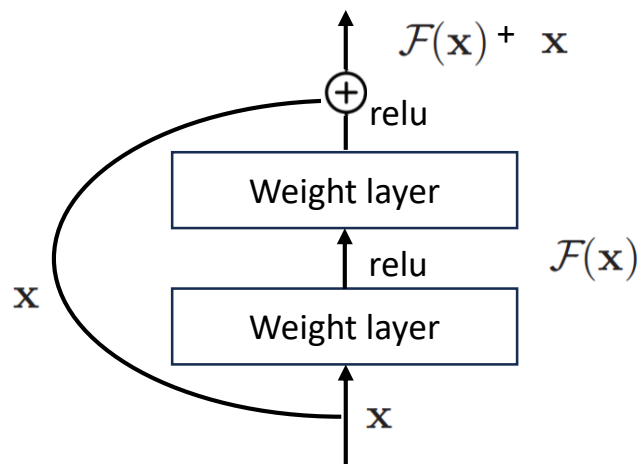


- 단어의 문맥적 중요도
- 단어마다의 연관성 포함
- 마지막 단어 집중 방지

모델의 문장 맥락 이해 과정

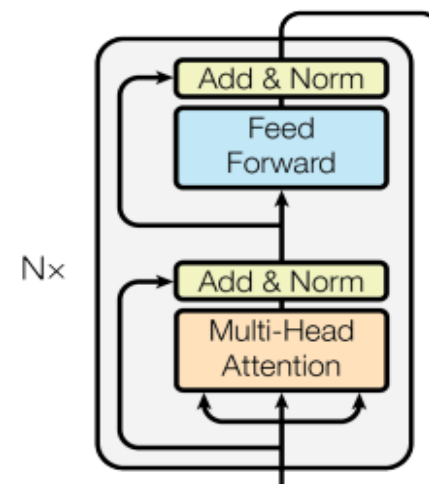
▪ Skip Connection ⇔ Resnet에서의 Residual 과 동일

- 이전 층의 정보를 연결하는 개념



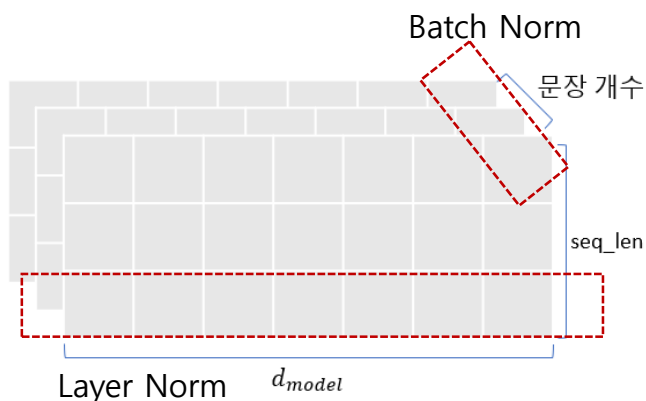
출력 결과물 : 입력 + 모델 출력

1. Gradient Vanishing 문제 해결
2. 안정적인 학습 (해당 논문 기준)



▪ Layer Normalization

- 데이터 샘플 단위 통계량 계산 (d_{model} 별 정규화)



특징	Layer Normalization	Batch Normalization
정규화 축	하나의 레이어(특성)	배치 차원(입력 샘플들)
독립성	배치 크기와 독립적	배치 크기에 의존적
적합한 사용 사례	시퀀스 데이터(RNN, 트랜스포머 등)	이미지 데이터(CNN 등)
학습 시 안정성	모든 입력에 대해 정규화	배치가 작은 경우 효과가 제한적
시간 축 정규화	가능 (시퀀스 모델에서 유용)	어려움

Encoder 총 과정

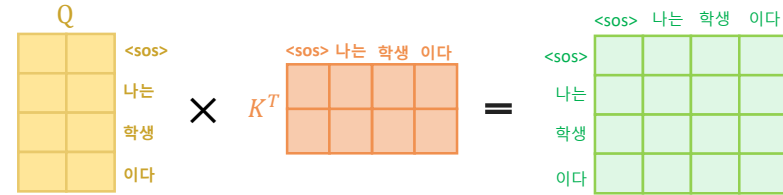
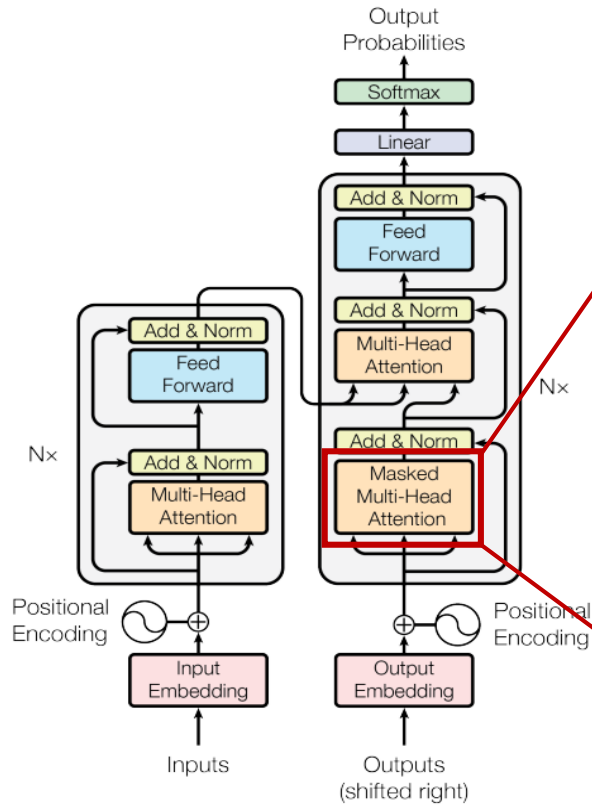
The encoder is composed of a stack of $N = 6$ identical layers.



Decoder (1)



Self Attention (Decoder)



자기 자신보다 미래의 단어를 참고 못하게



연산과정은 Encoder와 동일
&
Multi-Head Attention

1. Scaled Dot-Product Attention

Encoder와 동일한 Self-Attention을 기법을 적용하여 Attention Value Matrix 계산

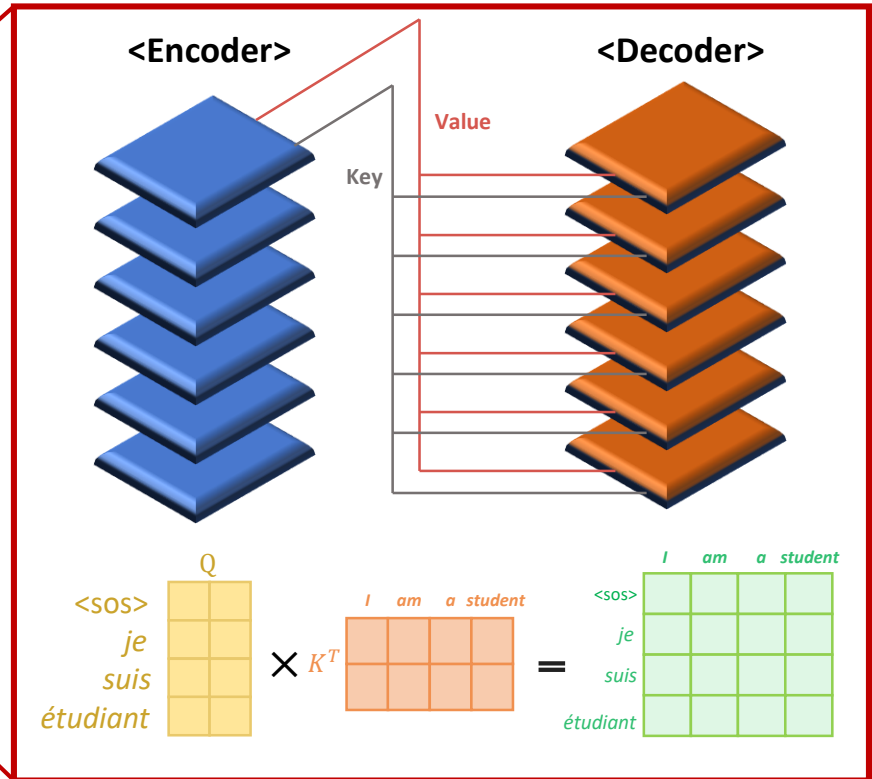
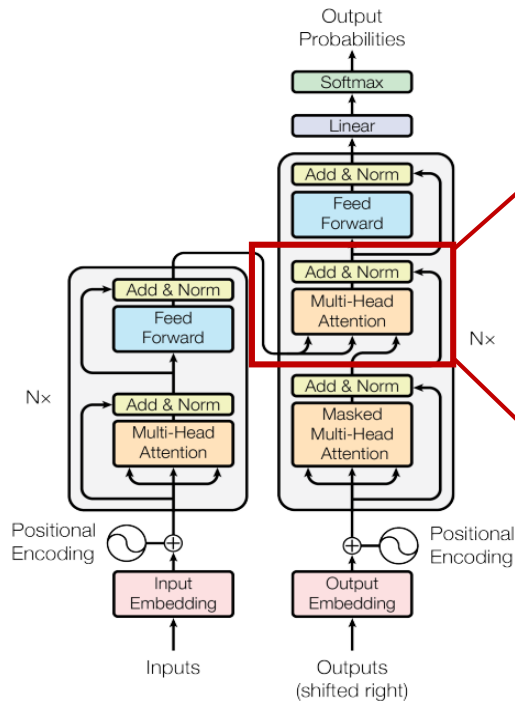
2. Look-Ahead Mask

각 단계에서 생성된 단어들이 미래의 단어의 영향을 주지 않도록 설정

Decoder (2)



Attention (Encoder - Decoder)



1. Query, Key, Value 다름

Query는 Decoder의 Matrix이고 Value, Key는 Encoder의 Matrix로 **Self Attention이 아님**

2. 정보 추출

인코더의 출력 전체에 주목하여, 생성 중인 단어와 관련된 중요한 정보 추출함

3. Attention 연산

Multi-Head Attention을 수행하는 과정은 Encoder의 연산 과정과 동일

[참고] Masked MHA



▪ Encoder와 Decoder의 차이점

Encoder : 입력 전체를 한번에 처리

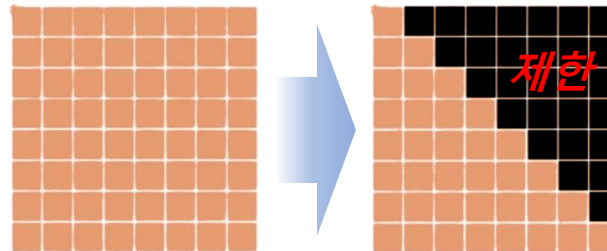
Decoder : **Next token prediction** 방식 (한 단어를 예측하고 이를 기반으로 다음 단어를 예측)

→ 미래 정보에 대한 접근 제한

→ **Masking** 처리 필요

▪ Next token prediction

모델에 일련의 토큰이 주어지고, 다음 토큰을 예측해야 하는 자기 지도 학습 기술



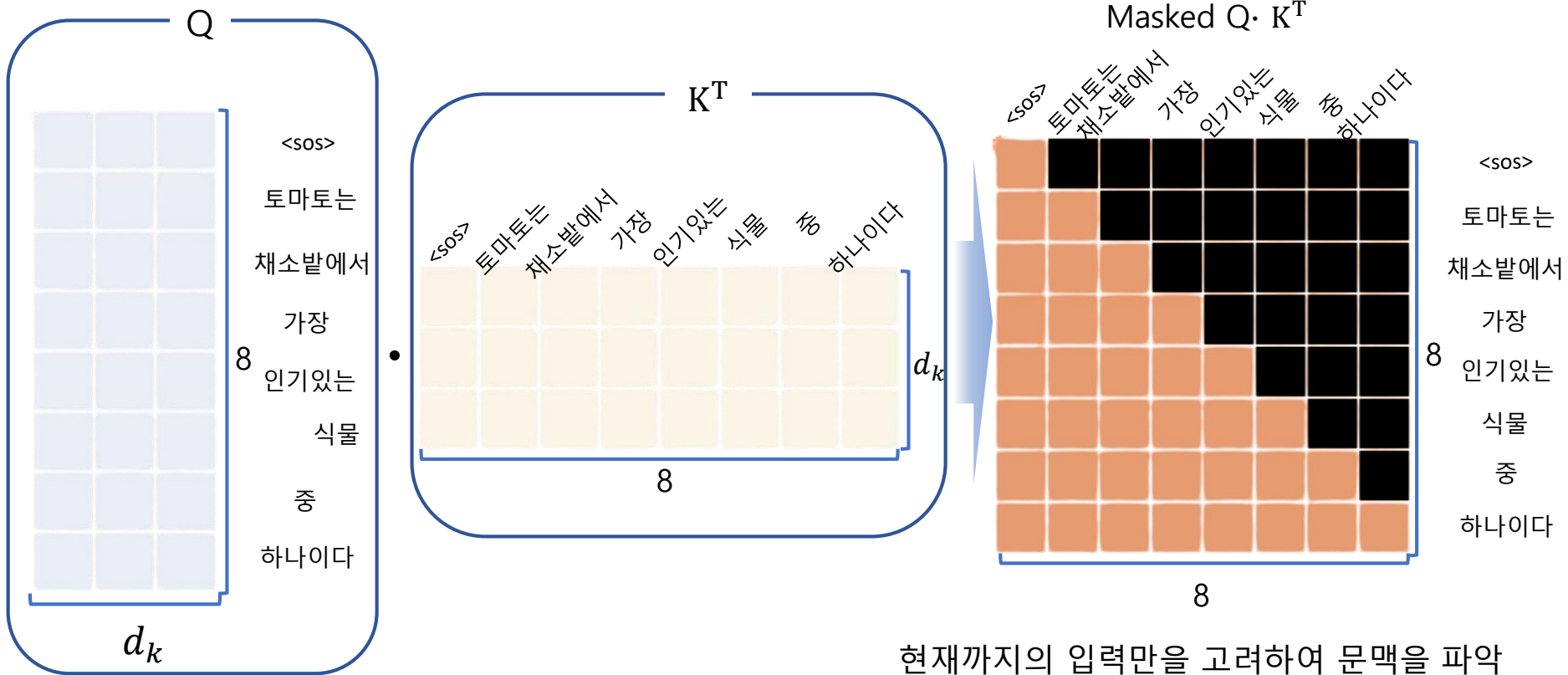
미래 정보에 대해 접근 제한 필요

[참고] Masked MHA



Case: train

미래 정보 참조를 막기 위해 적용 : **Look-Ahead Mask**

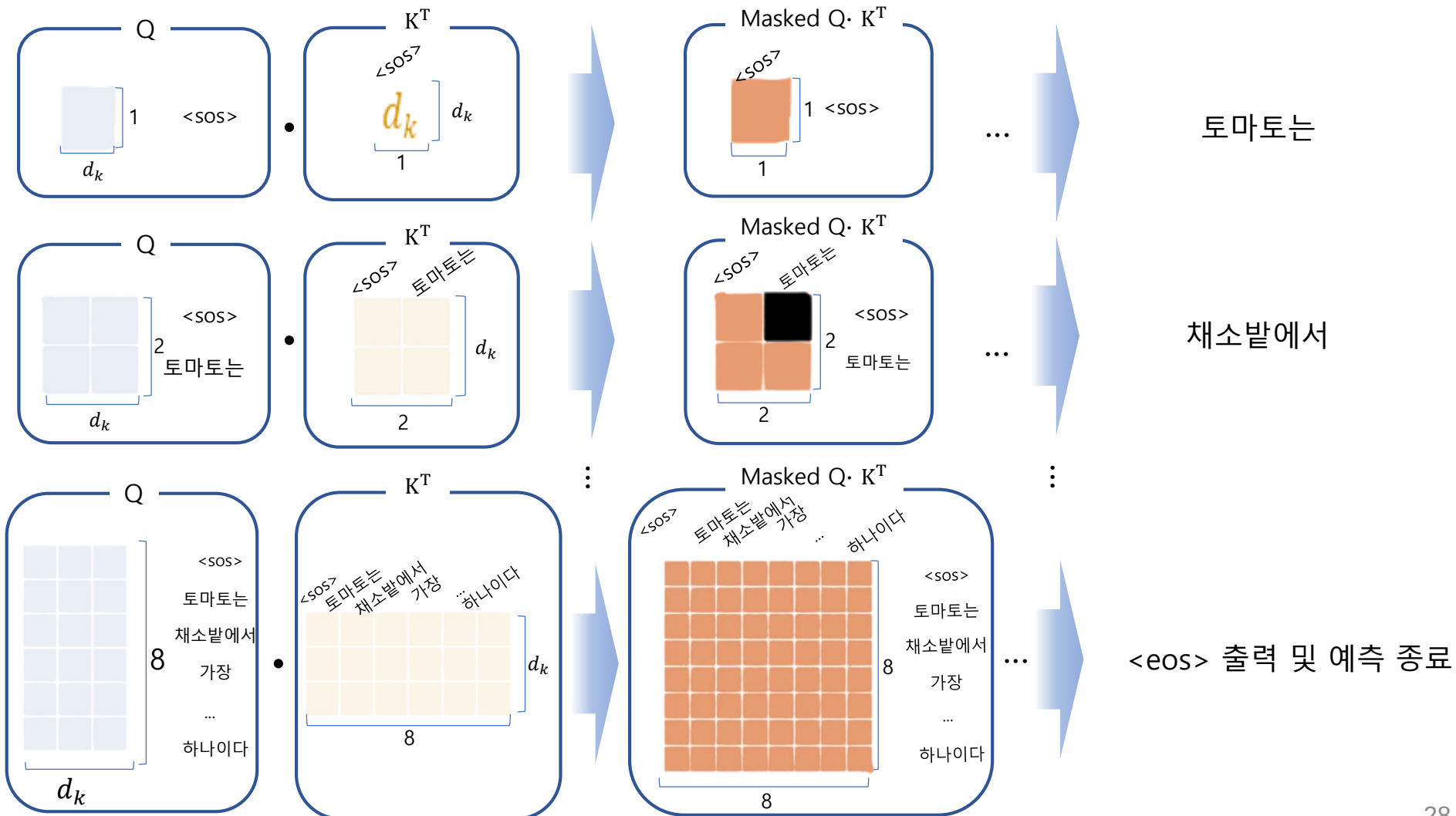


[참고] Masked MHA



Case: test

Testing에서는 <sos>로 Decoding을 시작하고, <eos>가 나올 때까지 단어를 추가하며 반복



[참고] Decoder



▪ Encoder-Decoder MHA

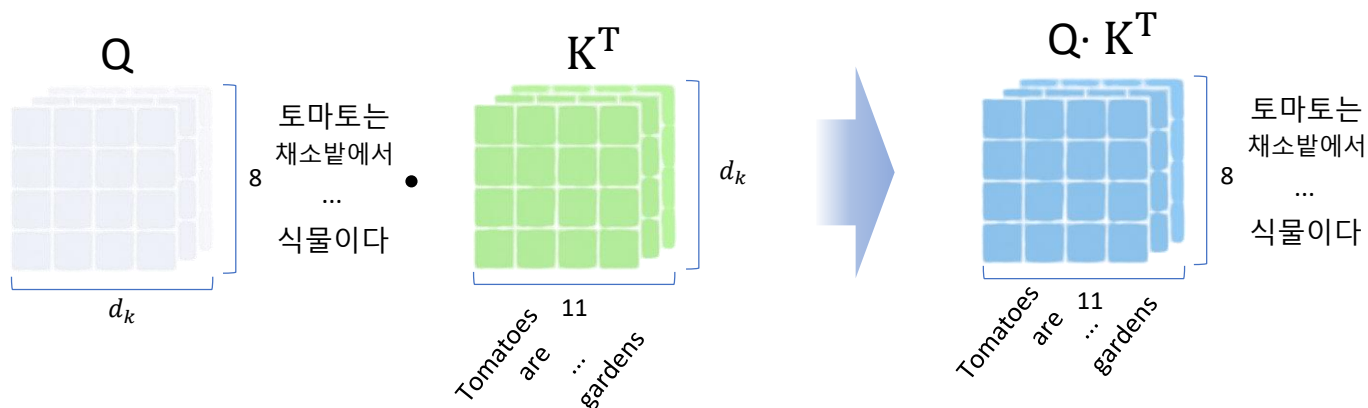
- Encoder-Decoder Attention은

Encoder에 대한 출력을 K, V로 Decoder Masked MHA에 대한 출력을 Q로 받음

-> Cross-Attention

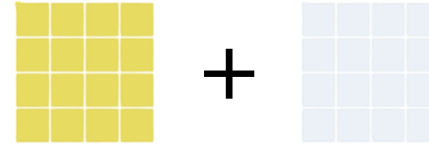
- Encoder의 출력에 대한 문맥이 고려된 Key와

Masked MHA의 출력(이전 Decoder 출력)에 대한 문맥이 고려된 Query의 내적을 이후 Value와의 내적을 통해 집중해야 할 단어를 확인



■ 전체적인 Decoder test 과정

- 입력 방식은 동일하나 <sos> 이후 <eos>가 출력될 때 까지 반복 수행



(모든 디코딩 시점에서 동일한 입력 (인코더 output) 사용)

